

OS-Lab3

思考题

Thinking3.1

`e->env_pgdir[PDX(UVPT)]`：找到 UVPT 固定虚拟地址，对应的页目录数组下标，也就是页目录里的某一个 PDE 表项位置

`PADDR(e->env_pgdir) | PTE_V`：页目录本体的物理页框号和有效权限标志位拼接，组装成合法的页目录项 PDE 的值

最终行为：把页目录自己的物理地址，写入到页目录自身、UVPT 索引对应的那个表项里，完成页目录自映射。

Thinking3.2

`elf_load_seg` 的调用：

```
// in env.c -> load_icode
ELF_FOREACH_PHDR_OFF (ph_off, ehdr) {
    Elf32_Phdr *ph = (Elf32_Phdr *)(binary + ph_off);
    if (ph->p_type == PT_LOAD) {
        panic_on(elf_load_seg(ph, binary + ph->p_offset, load_icode_mapper, e));
    }
}
```

`elf_load_seg`：

```

// in elfloader.c
int elf_load_seg(Elf32_Phdr *ph, const void *bin, elf_mapper_t map_page, void *data) {
// ...
    /* Step 1: load all content of bin into memory. */
    for (i = offset ? MIN(bin_size, PAGE_SIZE - offset) : 0; i < bin_size; i += PAGE_SIZE) {
        if ((r = map_page(data, va + i, 0, perm, bin + i, MIN(bin_size - i, PAGE_SIZE))) != 0) {
            return r;
        }
    }

    /* Step 2: alloc pages to reach `sgsize` when `bin_size` < `sgsize`. */
    while (i < sgsz) {
        if ((r = map_page(data, va + i, 0, perm, NULL, MIN(sgsz - i, PAGE_SIZE))) != 0) {
            return r;
        }
        i += PAGE_SIZE;
    }
// ...
}

```

传入 data 的是当前进程的控制块 Env，作为传给回调函数 map_page 的上下文参数，本身不被 elf_load_seg 使用。其作用是让 map_page 知道要给那个进程建立页表映射。

不能去掉，否则 map_page 不知道映射给哪个进程，会造成多进程环境下地址空间混乱。

Thinking3.3

```
int elf_load_seg(Elf32_Phdr *ph, const void *bin, elf_mapper_t map_page, void *data) {
    u_long va = ph->p_vaddr;
    size_t bin_size = ph->p_filesz;
    size_t sgsz = ph->p_memsz;
    u_int perm = PTE_V;
    if (ph->p_flags & PF_W) {
        perm |= PTE_D;
    }

    int r;
    size_t i;
    u_long offset = va - ROUNDDOWN(va, PAGE_SIZE);
    if (offset != 0) {
        if ((r = map_page(data, va, offset, perm, bin,
            MIN(bin_size, PAGE_SIZE - offset))) != 0) {
            return r;
        }
    }

    /* Step 1: load all content of bin into memory. */
    for (i = offset ? MIN(bin_size, PAGE_SIZE - offset) : 0; i < bin_size; i += PAGE_SIZE) {
        if ((r = map_page(data, va + i, 0, perm, bin + i, MIN(bin_size - i, PAGE_SIZE))) != 0) {
            return r;
        }
    }

    /* Step 2: alloc pages to reach `sgsz` when `bin_size` < `sgsz`. */
    while (i < sgsz) {
        if ((r = map_page(data, va + i, 0, perm, NULL, MIN(sgsz - i, PAGE_SIZE))) != 0) {
            return r;
        }
        i += PAGE_SIZE;
    }
    return 0;
}
```

考虑了以下特殊情况：

- 段头页不对齐的段，通过Offset将第一个不完整的页进行映射。
- 文件内容为占满整页，通过MIN保证写入内容长度

- 文件大小小于内存大小，仍继续映射，用NULL填满内存

Thinking3.4

指导书：

这里的env_tf.cp0_epc字段指示了进程恢复运行时PC应恢复到的位置。我们要运行的进程的代码段预先被载入到了内存中，且程序入口为e_entry，当我们运行进程时，CPU将自动从PC所指的位置开始执行二进制码。

CPU执行过程种都是虚拟地址，因此env_tf.cp0_epc是虚拟地址。

Thinking3.5

handle_int :

```
// in genex.S
NESTED(handle_int, TF_SIZE, zero)
    mfc0    t0, CP0_CAUSE
    mfc0    t2, CP0_STATUS
    and     t0, t2
    andi    t1, t0, STATUS_IM7
    bnez    t1, timer_irq
timer_irq:
    li      a0, 0
    j       schedule
END(handle_int)
```

do_tlb_mod , do_tlb_refill :

```
.macro BUILD_HANDLER exception handler
NESTED(handle_\exception, TF_SIZE + 8, zero)
    move    a0, sp
    addiu   sp, sp, -8
    jal     \handler
    addiu   sp, sp, 8
    j       ret_from_exception
END(handle_\exception)
.endm
```

```
BUILD_HANDLER tlb do_tlb_refill
```

```
#if !defined(LAB) || LAB >= 4
BUILD_HANDLER mod do_tlb_mod
BUILD_HANDLER sys do_syscall
#endif
```

```
BUILD_HANDLER reserved do_reserved
```

```
void do_tlb_mod(struct Trapframe *tf) {
    struct Trapframe tmp_tf = *tf;
    if (tf->regs[29] < USTACKTOP || tf->regs[29] >= UXSTACKTOP) {
        tf->regs[29] = UXSTACKTOP;
    }
    tf->regs[29] -= sizeof(struct Trapframe);
    *(struct Trapframe *)tf->regs[29] = tmp_tf;
    Pte *pte;
    page_lookup(cur_pgdir, tf->cp0_badvaddr, &pte);
    if (curenv->env_user_tlb_mod_entry) {
        tf->regs[4] = tf->regs[29];
        tf->regs[29] -= sizeof(tf->regs[4]);
        tf->cp0_epc = curenv->env_user_tlb_mod_entry;
    } else {
        panic("TLB Mod but no user handler registered");
    }
}
```

```

NESTED(do_tlb_refill, 24, zero)
    mfc0    a1, CP0_BADVADDR
    mfc0    a2, CP0_ENTRYHI
    andi    a2, a2, 0xff
.globl do_tlb_refill_call;
do_tlb_refill_call:
    addi    sp, sp, -24
    sw      ra, 20(sp)
    addi    a0, sp, 12
    jal     _do_tlb_refill
    lw      a0, 12(sp)
    lw      a1, 16(sp)
    lw      ra, 20(sp)
    addi    sp, sp, 24
    mtc0    a0, CP0_ENTRYLO0
    mtc0    a1, CP0_ENTRYLO1
    nop
    tlbwr
    jr      ra
END(do_tlb_refill)

```

Thinking3.6

内核启动入口 entry.S 中，会首先清空 CP0 Status 寄存器，全局关闭所有中断（包含时钟中断），保证初始化、页表搭建、环境初始化等临界代码不会被干扰。

genex.S 通用异常入口硬件自动屏蔽中断；汇编入口软件主动关闭全局中断。

内核执行不可被抢占的临界操作（页表修改、进程链表操作、锁操作）时，主动调用 disable_irq（位于 env_asm.S）关闭时钟与全局中断。

env_asm.S 的进程切换汇编代码，在保存 / 恢复进程上下文的整个过程中，保持时钟中断关闭。

总结：

- 时钟中断开启：线程启动，用户态正常运行中，线程异常处理结束。
- 时钟中断结束：线程陷入中断/异常。

Thinking3.7

Step 1: 时钟中断进入 exc_gen_entry（在 kernel1.ld 中调用，即由硬件实现）

Step 2: 进入内核态，检测异常信息，通过 handle_int 进行跳转，调用 schedule 函数

```

void schedule(int yield) {
    static int count = 0;
    struct Env *e = curenv;
    if (e == NULL || count == 0 || e->env_status != ENV_RUNNABLE || yield) {
        if (e != NULL && e->env_status == ENV_RUNNABLE) {
            TAILQ_REMOVE(&env_sched_list, e, env_sched_link);
            TAILQ_INSERT_TAIL(&env_sched_list, e, env_sched_link);
        }
        e = TAILQ_FIRST(&env_sched_list);
        if (e == NULL) {
            panic("schedule: no runnable envs\n"); //Panic if the list is empty
        }
        count = e->env_pri;
    }
    count--;
    env_run(e);
}

```

Step 3: 通过 schedule 函数实现进程切换

难点分析

进程

进程建立

使用 env_init 初始化进程管理系统的数据结构，其中

- 用 env_free_list 初始化空闲进程列表，用 env_sched_list 初始化调度进程队列。
- 遍历 envs 数组，设置状态为 ENV_FREE ，插入到 env_free_list 中。
- 分配一个页作为基础页目录 base_pgdir 。

使用 env_setup_vm 为单个进程e初始化用户地址空间。

使用 env_alloc 分配并初始化一个新的进程。

使用 load_icode_mapper 和 load_icode 加载可执行文件，初始化进程入口。

使用 env_create 创建进程。

进程启动

使用 `env_pop_tf` 恢复用户态执行上下文

```
mtc0    a1, CP0_ENTRYHI
move     sp, a0
RESET_KCLOCK
RESTORE_ALL
eret
```

使用 `env_run` 切换到目标用户进程并执行。

其中KSTACKTOP是内核栈的顶部地址，`(struct Trapframe *)KSTACKTOP - 1`指向内核栈中保存的Trapframe讲内核栈中的Trapframe复制到`curenv->env_tf`，以便后续恢复。

进程结束

使用 `env_free` 释放一个线程：遍历页表，释放物理页，释放页表和页目录，释放ASID，刷新TLB，放回空闲列表。

使用 `env_destroy` 摧毁进程。

异常中断

硬件识别异常后，把PC跳转到异常入口，保存上下文，进入内核态，得到异常类型，跳转到异常处理函数。

`handle_int`：

```
// in genex.S
NESTED(handle_int, TF_SIZE, zero)
    mfc0    t0, CP0_CAUSE
    mfc0    t2, CP0_STATUS
    and     t0, t2
    andi    t1, t0, STATUS_IM7
    bnez    t1, timer_irq
timer_irq:
    li      a0, 0
    j       schedule
END(handle_int)
```

`do_tlb_mod` , `do_tlb_refill`：


```

.macro BUILD_HANDLER exception handler
NESTED(handle_\exception, TF_SIZE + 8, zero)
    move    a0, sp
    addiu   sp, sp, -8
    jal     \handler
    addiu   sp, sp, 8
    j       ret_from_exception
END(handle_\exception)
.endm

```

```

BUILD_HANDLER tlb do_tlb_refill

```

```

#if !defined(LAB) || LAB >= 4
BUILD_HANDLER mod do_tlb_mod
BUILD_HANDLER sys do_syscall
#endif

```

```

BUILD_HANDLER reserved do_reserved

```

```

void do_tlb_mod(struct Trapframe *tf) {
    struct Trapframe tmp_tf = *tf;
    if (tf->regs[29] < USTACKTOP || tf->regs[29] >= UXSTACKTOP) {
        tf->regs[29] = UXSTACKTOP;
    }
    tf->regs[29] -= sizeof(struct Trapframe);
    *(struct Trapframe *)tf->regs[29] = tmp_tf;
    Pte *pte;
    page_lookup(cur_pgdir, tf->cp0_badvaddr, &pte);
    if (curenv->env_user_tlb_mod_entry) {
        tf->regs[4] = tf->regs[29];
        tf->regs[29] -= sizeof(tf->regs[4]);
        tf->cp0_epc = curenv->env_user_tlb_mod_entry;
    } else {
        panic("TLB Mod but no user handler registered");
    }
}

```

```

NESTED(do_tlb_refill, 24, zero)
    mfc0    a1, CP0_BADVADDR
    mfc0    a2, CP0_ENTRYHI
    andi    a2, a2, 0xff
.globl do_tlb_refill_call;
do_tlb_refill_call:
    addi    sp, sp, -24
    sw      ra, 20(sp)
    addi    a0, sp, 12
    jal     _do_tlb_refill
    lw      a0, 12(sp)
    lw      a1, 16(sp)
    lw      ra, 20(sp)
    addi    sp, sp, 24
    mtc0    a0, CP0_ENTRYLO0
    mtc0    a1, CP0_ENTRYLO1
    nop
    tlbwr
    jr      ra
END(do_tlb_refill)

```

实验体会

经过了Lab2的学习，对许多内置宏有了更深的理解，阅读Lab3的代码更加简单了。

相比Lab2，Lab3围绕进程作为核心，研究主题更加固定且明确，进程的创建，启动，运行，结束都十分按部就班，顺着指导书阅读很快就能看懂。相对比较特殊的是进程的异常中断和进程调度部分。

进程的异常中断紧接CO-P7的内容，如果对CO的东西还有印象的话会更好理解中断的代码。进程调度部分非常灵活，exam主要考的也是进程调度，但总之万变不离其宗，对那几个列表和进程的创建流程理解到位还是比较公式的。

总之Lab3无论是学习过程还是上机难度都还是比较亲切的。